

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

ENTREGA 3A

Layouts y Sidebar Dinámico

Proyecto: gestión-académica

Fase 1 — Fundación | Prerrequisito: Entregas 1A, 1B, 2A y 2B completadas

Abril 2026

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Configurar el layout raíz de Next.js con fuentes personalizadas y metadatos.
- Instalar y configurar el componente Sidebar de shadcn/ui.
- Construir un sidebar con navegación dinámica que cambia según el rol del usuario.
- Implementar el patrón de composición SidebarProvider + SidebarInset para el layout del dashboard.
- Comprender la jerarquía de layouts en Next.js App Router.

Prerrequisitos

- ✓ Entregas 1A, 1B, 2A y 2B completadas
- ✓ Flujo de login/registro funcional
- ✓ Al menos un usuario de prueba creado en Supabase

Teoría: Jerarquía de Layouts en Next.js

En Next.js App Router, los layouts se anidan automáticamente. Cada carpeta puede tener su propio layout.tsx que envuelve a sus páginas hijas. Nuestra aplicación tendrá tres niveles de layout:

Jerarquía de layouts

JERARQUÍA DE LAYOUTS:

```

app/layout.tsx           ← Layout raíz (fuentes, <html>, <body>)
├── app/(auth)/layout.tsx ← Layout auth (centrado, sin sidebar)
│   ├── /login/page.tsx
│   └── /registro/page.tsx
└── app/(dashboard)/layout.tsx ← Layout dashboard (sidebar + header)
    ├── /estudiante/page.tsx
    ├── /docente/page.tsx
    └── /admin/page.tsx
  
```

Cada página se renderiza así:

```

<RootLayout>           ← Fuentes, tema, <html>
  <DashboardLayout>     ← Sidebar + header
    <Page />           ← Contenido específico
  </DashboardLayout>
</RootLayout>
  
```

Regla clave: el layout raíz (app/layout.tsx) es el Único que debe contener las etiquetas `<html>` y `<body>`. Los demás layouts solo definen estructura interna.

Instalar los componentes necesarios de shadcn/ui

El Sidebar de shadcn/ui es un componente compuesto que requiere varias piezas. Detiene el servidor e instala:

Terminal

```
npx shadcn@latest add sidebar
npx shadcn@latest add separator
npx shadcn@latest add dropdown-menu
npx shadcn@latest add avatar
npx shadcn@latest add tooltip
```

Estos componentes proporcionan:

Componente	Uso en nuestra app
<code>sidebar</code>	Navegación lateral colapsable con soporte móvil automático
<code>separator</code>	Líneas divisoras entre secciones del sidebar y header
<code>dropdown-menu</code>	Menú desplegable para el usuario (logout, perfil)
<code>avatar</code>	Foto de perfil del usuario en el sidebar y header
<code>tooltip</code>	Tooltips cuando el sidebar está colapsado (muestra el nombre del ítem)

Configurar el layout raíz

El layout raíz define la estructura HTML base, las fuentes tipográficas y los metadatos de la aplicación. Es compartido por TODAS las páginas.

Abre **app/layout.tsx** y reemplaza todo su contenido con:

Archivo: app/layout.tsx

```
import type { Metadata } from "next"
import { Inter } from "next/font/google"
import { Toaster } from "@components/ui/sonner"
import "./globals.css"

const inter = Inter({
  subsets: ["latin"],
  variable: "--font-sans",
})
```

```
export const metadata: Metadata = {
  title: "Gestión Académica",
  description: "Sistema de gestión universitaria – UCB",
}

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="es" suppressHydrationWarning>
      <body className={` ${inter.variable} font-sans antialiased`} >
        {children}
        <Toaster richColors position="top-right" />
      </body>
    </html>
  )
}
```

Desglose del layout raíz

Inter font: Next.js optimiza las fuentes de Google automáticamente. La fuente se descarga en build time y se sirve localmente (sin peticiones a Google en runtime). La variable CSS `--font-sans` está disponible en toda la app.

Metadata: El objeto metadata define el título y descripción de la página. Next.js lo convierte automáticamente en etiquetas `<title>` y `<meta>` en el `<head>`.

lang="es": Define el idioma como español, importante para accesibilidad y SEO.

Toaster: El componente de Sonner (instalado en la Entrega 1B) muestra notificaciones toast. Se coloca aquí para que esté disponible en toda la app sin importarlo en cada página.

suppressHydrationWarning: Evita warnings cuando hay diferencias entre servidor y cliente por extensiones del navegador que modifican el HTML.

Crear la configuración de navegación por rol

Antes de construir el sidebar, definimos qué items de menú ve cada rol. Este archivo centraliza la navegación y hace fácil agregar o modificar items sin tocar el componente visual.

Crea el archivo `lib/navigation.ts`:

```
Archivo: lib/navigation.ts
import {
  BookOpen,
  ClipboardList,
  GraduationCap,
```

```

    LayoutDashboard,
    Settings,
    Users,
    UserCircle,
  } from "lucide-react"

export type NavItem = {
  title: string
  url: string
  icon: typeof LayoutDashboard
}

export type NavGroup = {
  label: string
  items: NavItem[]
}

// Navegación para cada rol
export const navigationConfig: Record<string, NavGroup[]> = {
  estudiante: [
    {
      label: "Principal",
      items: [
        { title: "Dashboard", url: "/estudiante", icon: LayoutDashboard },
        { title: "Materias", url: "/estudiante/materias", icon: BookOpen },
        {
          title: "Mis Inscripciones",
          url: "/estudiante/inscripciones",
          icon: ClipboardList,
        },
      ],
    },
    {
      label: "Cuenta",
      items: [
        { title: "Mi Perfil", url: "/estudiante/perfil", icon: UserCircle },
      ],
    },
  ],
  docente: [
    {
      label: "Principal",
      items: [
        { title: "Dashboard", url: "/docente", icon: LayoutDashboard },
        { title: "Mis Materias", url: "/docente/materias", icon: BookOpen },
      ],
    },
    {
      label: "Cuenta",
      items: [
        { title: "Mi Perfil", url: "/docente/perfil", icon: UserCircle },
      ],
    },
  ],
}

```

```

    },
  ],
  admin: [
    {
      label: "Principal",
      items: [
        { title: "Dashboard", url: "/admin", icon: LayoutDashboard },
        { title: "Usuarios", url: "/admin/usuarios", icon: Users },
        { title: "Materias", url: "/admin/materias", icon: BookOpen },
        {
          title: "Inscripciones",
          url: "/admin/inscripciones",
          icon: ClipboardList,
        },
      ],
    },
  ],
  {
    label: "Sistema",
    items: [
      { title: "Configuración", url: "/admin/config", icon: Settings },
    ],
  },
],
}

```

Tip: Lucide React

lucide-react viene incluido con shadcn/ui. Ofrece más de 1500 iconos. Puedes explorar el catálogo completo en <https://lucide.dev/icons/>

¿Por qué un archivo separado? Separar la configuración de navegación del componente visual sigue el principio de **separación de responsabilidades**. Si mañana quieres agregar una sección "Reportes" al admin, solo editas este archivo sin tocar el sidebar.

Crear el componente AppSidebar

Este es el componente central del sidebar. Lee el rol del usuario actual y renderiza los items de navegación correspondientes usando la configuración que creamos en el paso anterior.

Crea el archivo **components/dashboard/app-sidebar.tsx**:

```

Archivo: components/dashboard/app-sidebar.tsx
"use client"

import Link from "next/link"
import { usePathname } from "next/navigation"
import { GraduationCap } from "lucide-react"

import {
  Sidebar,

```

```

SidebarContent,
SidebarGroup,
SidebarGroupLabel,
SidebarGroupContent,
SidebarMenu,
SidebarMenuItem,
SidebarMenuButton,
SidebarHeader,
SidebarFooter,
SidebarRail,
} from "@components/ui/sidebar"

import { navigationConfig, type NavGroup } from "@lib/navigation"

interface AppSidebarProps {
  rol: string
  nombreUsuario: string
}

export function AppSidebar({ rol, nombreUsuario }: AppSidebarProps) {
  const pathname = usePathname()

  // Obtener los grupos de navegación según el rol
  const navGroups: NavGroup[] = navigationConfig[rol] || []

  return (
    <Sidebar collapsible="icon">
      {/* — Header del sidebar: Logo + nombre — */}
      <SidebarHeader>
        <SidebarMenu>
          <SidebarMenuItem>
            <SidebarMenuButton size="lg" asChild>
              <Link href={`/${rol}`}>
                <div className="flex aspect-square size-8 items-center justify-center rounded-lg bg-primary text-primary-foreground">
                  <GraduationCap className="size-4" />
                </div>
                <div className="flex flex-col gap-0.5 leading-none">
                  <span className="font-semibold">Gestión Académica</span>
                  <span className="text-xs text-muted-foreground">
                    {rol.charAt(0).toUpperCase() + rol.slice(1)}
                  </span>
                </div>
              </Link>
            </SidebarMenuButton>
          </SidebarMenuItem>
        </SidebarMenu>
      </SidebarHeader>

      {/* — Contenido: grupos de navegación — */}
      <SidebarContent>

```

```

    {navGroups.map((group) => (
      <SidebarGroup key={group.label}>
        <SidebarGroupLabel>{group.label}</SidebarGroupLabel>
        <SidebarGroupContent>
          <SidebarMenu>
            {group.items.map((item) => (
              <SidebarMenuItem key={item.url}>
                <SidebarMenuButton
                  asChild
                  isActive={pathname === item.url}
                  tooltip={item.title}
                >
                  <Link href={item.url}>
                    <item.icon />
                    <span>{item.title}</span>
                  </Link>
                </SidebarMenuButton>
              </SidebarMenuItem>
            ))}
          </SidebarMenu>
        </SidebarGroupContent>
      </SidebarGroup>
    ))}
  </SidebarContent>

  {/* — Footer: nombre del usuario — */}
  <SidebarFooter>
    <SidebarMenu>
      <SidebarMenuItem>
        <SidebarMenuButton size="lg">
          <div className="flex aspect-square size-8 items-center justify-center
rounded-lg bg-muted">
            <span className="text-sm font-medium">
              {nombreUsuario.charAt(0).toUpperCase()}
            </span>
          </div>
          <div className="flex flex-col gap-0.5 leading-none">
            <span className="text-sm font-medium">{nombreUsuario}</span>
            <span className="text-xs text-muted-foreground">{rol}</span>
          </div>
        </SidebarMenuButton>
      </SidebarMenuItem>
    </SidebarMenu>
  </SidebarFooter>

  <SidebarRail />
</Sidebar>
)
}

```

Desglose del AppSidebar

Props: Recibe `rol` y `nombreUsuario` desde el layout padre. El layout los obtiene de Supabase en el servidor.

`collapsible="icon"`: Permite que el sidebar se colapse a un modo de solo íconos. El usuario puede alternar con `Ctrl+B` o el botón `SidebarTrigger`.

`isActive`: Compara la URL actual (`pathname`) con la URL del item. Si coinciden, el item se resalta visualmente.

`tooltip`: Cuando el sidebar está colapsado, muestra un tooltip con el nombre del item al pasar el mouse.

`<item.icon />`: Los iconos se pasan como componentes React en la configuración. Esta línea los renderiza dinámicamente.

`SidebarRail`: Añade una barra delgada en el borde del sidebar que permite redimensionarlo arrastrando.

Crear el layout del dashboard

El layout del dashboard envuelve todas las páginas protegidas. Obtiene los datos del usuario desde el servidor y se los pasa al `AppSidebar`.

Crea el archivo `app/(dashboard)/layout.tsx`:

```
Archivo: app/(dashboard)/layout.tsx
import { redirect } from "next/navigation"
import { createClient } from "@lib/supabase/server"

import { SidebarProvider, SidebarInset, SidebarTrigger } from
"@components/ui/sidebar"
import { Separator } from "@components/ui/separator"
import { AppSidebar } from "@components/dashboard/app-sidebar"

export default async function DashboardLayout({
  children,
}): {
  children: React.ReactNode
}) {
  // 1. Obtener el usuario autenticado
  const supabase = await createClient()

  const {
    data: { user },
  } = await supabase.auth.getUser()

  // Si no hay usuario, redirigir a login
  if (!user) {
    redirect("/login")
  }
}
```

```

}

// 2. Obtener el perfil del usuario (rol, nombre)
const { data: perfil } = await supabase
  .from("usuarios_perfil")
  .select("nombre_completo, rol")
  .eq("id", user.id)
  .single()

if (!perfil) {
  redirect("/login")
}

return (
  <SidebarProvider>
    <AppSidebar
      rol={perfil.rol}
      nombreUsuario={perfil.nombre_completo}
    />
    <SidebarInset>
      {/* — Header del contenido — */}
      <header className="flex h-16 shrink-0 items-center gap-2 border-b px-4">
        <SidebarTrigger className="-ml-1" />
        <Separator
          orientation="vertical"
          className="mr-2 data-[orientation=vertical]:h-4"
        />
        <span className="text-sm text-muted-foreground">
          {perfil.rol.charAt(0).toUpperCase() + perfil.rol.slice(1)}
        </span>
      </header>

      {/* — Contenido principal — */}
      <main className="flex-1 p-6">
        {children}
      </main>
    </SidebarInset>
  </SidebarProvider>
)
}

```

Desglose del layout del dashboard

async function: Este layout es un **Server Component asíncrono**. Puede hacer await directamente para consultar Supabase. Esto es imposible en Client Components.

Doble protección: Aunque proxy.ts ya protege las rutas, verificamos de nuevo aquí con `redirect("/login")`. Es defensa en profundidad — si alguien bypasea proxy.ts, el layout lo detiene.

SidebarProvider: Envuelve todo el layout y proporciona el contexto necesario para que el sidebar funcione (estado colapsado/expandido, móvil/desktop).

SidebarInset: Contiene el área de contenido principal. Se ajusta automáticamente cuando el sidebar se colapsa o expande.

SidebarTrigger: Botón hamburguesa que alterna el sidebar. En móvil, abre un panel deslizable (Sheet).

Nota: Flujo de datos server → client

El layout (Server Component) obtiene rol y nombre de Supabase. Luego se los pasa como props al AppSidebar (Client Component). Este patrón de "fetch en el servidor, pasar props al cliente" es el estándar en Next.js 16.

Crear las páginas placeholder de cada dashboard

Necesitamos al menos una página por cada rol para verificar que el layout y la navegación funcionan. Por ahora serán placeholders simples.

Crea `app/(dashboard)/estudiante/page.tsx`:

```
Archivo: app/(dashboard)/estudiante/page.tsx
export default function EstudianteDashboard() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Dashboard Estudiante</h1>
      <p className="text-muted-foreground">
        Bienvenido al panel de estudiante. Aquí verás tus materias,
        inscripciones y calificaciones.
      </p>
    </div>
  )
}
```

Crea `app/(dashboard)/docente/page.tsx`:

```
Archivo: app/(dashboard)/docente/page.tsx
export default function DocenteDashboard() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Dashboard Docente</h1>
      <p className="text-muted-foreground">
        Bienvenido al panel de docente. Aquí gestionarás tus materias
        y calificaciones de estudiantes.
      </p>
    </div>
  )
}
```

Crea `app/(dashboard)/admin/page.tsx`:

```
Archivo: app/(dashboard)/admin/page.tsx
export default function AdminDashboard() {
  return (
    <div className="space-y-4">
      <h1 className="text-3xl font-bold">Dashboard Administrador</h1>
      <p className="text-muted-foreground">
        Bienvenido al panel de administración. Aquí gestionarás
        usuarios, materias e inscripciones del sistema.
      </p>
    </div>
  )
}
```

Actualizar la página raíz

La página raíz (ruta "/") debe redirigir automáticamente: si el usuario está autenticado, va a su dashboard; si no, va al login.

Actualiza `app/page.tsx`:

```
Archivo: app/page.tsx
import { redirect } from "next/navigation"
import { createClient } from "@/lib/supabase/server"

export default async function Home() {
  const supabase = await createClient()

  const {
    data: { user },
  } = await supabase.auth.getUser()

  if (!user) {
    redirect("/login")
  }

  // Si hay usuario, obtener su rol y redirigir
  const { data: perfil } = await supabase
    .from("usuarios_perfil")
    .select("rol")
    .eq("id", user.id)
    .single()

  if (perfil) {
    redirect(`/${perfil.rol}`)
  }

  redirect("/login")
}
```

}

Probar el layout completo

Es momento de ver la aplicación con sidebar funcionando:

1. Ejecuta **npm run dev**
2. Ve a <http://localhost:3000>. Deberías ser redirigido a /login.
3. Inicia sesión con tu usuario de prueba.
4. Deberías ver el dashboard con:
 - Sidebar a la izquierda con el logo y los items de navegación según tu rol.
 - Header con el botón hamburguesa y el nombre del rol.
 - Contenido principal con el texto del placeholder.
5. **Prueba colapsar el sidebar:** Haz clic en el botón hamburguesa o presiona Ctrl+B. El sidebar debe colapsar a modo de íconos.
6. **Prueba la navegación:** Haz clic en cada item del menú. Aunque las páginas mostrarán 404 (las crearemos después), el item activo debe resaltarse.
7. **Prueba responsivo:** Reduce el ancho del navegador. En pantallas pequeñas, el sidebar debe convertirse en un panel deslizable que se abre con el botón hamburguesa.

Tip: Prueba con diferentes roles

Cierra sesión e inicia con un usuario de diferente rol. El sidebar debe mostrar items distintos según el rol (el admin ve más opciones que el estudiante).

Verificación Final

Confirma todos estos puntos antes de avanzar a la Entrega 3B:

- ✓ `app/layout.tsx` tiene la fuente Inter, metadata y Toaster configurados
- ✓ `app/(dashboard)/layout.tsx` muestra el sidebar y header correctamente
- ✓ El sidebar muestra items diferentes según el rol (estudiante/docente/admin)
- ✓ El sidebar se colapsa a modo íconos con Ctrl+B
- ✓ El item activo se resalta en el sidebar
- ✓ En móvil, el sidebar es un panel deslizable
- ✓ La ruta "/" redirige al dashboard correcto según el rol

- ✓ Las páginas placeholder muestran su contenido dentro del layout

Ejercicio Propuesto

8. **Agrega un ítem al sidebar:** Añade un ítem "Calendario" al menú del estudiante en `lib/navigation.ts`. Usa el ícono Calendar de `lucide-react`. Verifica que aparece en el sidebar.
9. **Personaliza el header:** Agrega el nombre del usuario al header del dashboard (junto al rol). Pista: pasa `nombreUsuario` al header desde el layout.
10. **Investiga el componente Sidebar:** Abre `components/ui/sidebar.tsx` y busca la variable `SIDEBAR_WIDTH`. ¿Qué valor tiene? Cambia el ancho a 280px y observa el resultado.
11. **Pregunta de reflexión:** ¿Por qué el layout del dashboard es un Server Component pero el `AppSidebar` es un Client Component? ¿Qué pasaría si intentáramos usar `usePathname()` directamente en el layout?

Próximo: Entrega 3B

En la siguiente entrega completaremos la Fase 1 añadiendo el header con avatar y dropdown de usuario, la funcionalidad de logout, las páginas placeholder para todas las secciones, el hook `useUsuario` para acceder a los datos del usuario actual, y la API Route para actualizar el perfil.